L A P O R A N P R A K T I K U M

IMPLEMENTASI NEXT.JS DAN TAILWIND CSS DALAM PEMBUATAN LANDING PAGE WEBSITE

Dosen Pengampu:
Abdul Fikri, B.Sc, GradDip InnEntr.

Disusun Oleh:
Fadhlun (220212021)

FAKULTAS TARBIYAH DAN KEGURUAN
UNIVERSITAS ISLAM NEGERI AR-RANIRY
BANDA ACEH
2026/1447

DAFTAR ISI

BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan Praktikum 2
1.4 Manfaat Praktikum 2
1.5 Batasan Masalah 3
1.6 Sistematika Penulisan 3
BAB II LANDASAN TEORI DAN LANGKAH KERJA **4**
2.1 Landasan Teori 4
2.2 Alat dan Bahan 9
2.3 Langkah Kerja 10
BAB III HASIL DAN PEMBAHASAN **12**
3.1 Hasil Praktikum 12
3.2 Proses Pembuatan Aplikasi 12
3.3 Implementasi Landing Page 15
3.4 Pembahasan 19
BAB IV PENUTUP **22**
4.1 Kesimpulan 22
4.2 Saran 22
DAFTAR PUSTAKA **23**

DAFTAR TABEL

Tabel 1.1. Sistematika Penulisan Laporan..... 3
Tabel 2.1. File dan Folder Utama pada Proyek Next.js 5
Tabel 2.2. Perbandingan CSS Konvensional dan Tailwind CSS..... 6
Tabel 2.3. Kelompok Utility Class Tailwind CSS..... 6
Tabel 2.4. Breakpoint Responsif Tailwind CSS 7
Tabel 2.5. Bagian-Bagian Umum Landing Page 8
Tabel 2.6. Rujukan Penelitian yang Mendasari Praktikum 9
Tabel 2.7. Alat dan Bahan Praktikum 9
Tabel 2.8. Ringkasan Perintah yang Digunakan..... 11
Tabel 3.1. Hasil Pemeriksaan Lingkungan Pengembangan..... 13
Tabel 3.2. Class Tailwind CSS pada Tiap Bagian Landing Page..... 17
Tabel 3.3. Kesesuaian Hasil dengan Rancangan Landing Page 20
Tabel 3.4. Kendala yang Ditemui dan Penyelesaiannya..... 20
Tabel 3.5. Jawaban atas Rumusan Masalah 21

DAFTAR GAMBAR

Gambar 3.1. Pemeriksaan versi Node.js dan npm 13
Gambar 3.2. Struktur project Next.js pada Visual Studio Code 14
Gambar 3.3. Menjalankan aplikasi Next.js melalui terminal..... 15
Gambar 3.4. Tampilan navigasi dan hero section pada browser 18
Gambar 3.5. Tampilan bagian Tentang Saya pada browser..... 19
Gambar 3.6. Tampilan bagian Hobi dan kutipan motivasi pada browser 19

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah mendorong penggunaan website sebagai media penyampaian informasi dan layanan digital. Website tidak hanya berfungsi untuk menampilkan informasi, tetapi juga perlu memiliki tampilan yang menarik, terstruktur, dan mudah digunakan oleh pengguna. Salah satu bentuk halaman website yang sering digunakan adalah *landing page*, yaitu halaman yang menyajikan informasi utama mengenai suatu produk, layanan, atau organisasi. Dalam proses pembuatannya, diperlukan teknologi yang dapat membantu pengembang menyusun struktur halaman dan mengatur desain antarmuka secara efektif.

Next.js merupakan *framework* berbasis React yang dapat digunakan untuk membangun aplikasi web dengan struktur komponen yang terorganisasi. Dalam penelitian Maharani (2025) mengenai pengembangan website PT. Rantangin Digital Indonesia, Next.js digunakan bersama Tailwind CSS untuk mendukung proses pengembangan website. Penggunaan kedua teknologi tersebut relevan untuk dipelajari karena Next.js berperan dalam penyusunan halaman aplikasi, sedangkan Tailwind CSS membantu pengembang mengatur tampilan antarmuka melalui *utility class* yang dapat diterapkan langsung pada elemen halaman.

Penelitian Azhariyah dan Mukhlis (2024) membahas penggunaan Tailwind CSS dalam pengembangan bagian *front-end* website. Tailwind CSS menyediakan berbagai *utility class* untuk mengatur warna, ukuran teks, jarak, tata letak, dan tampilan responsif. Pendekatan tersebut mempermudah proses modifikasi desain tanpa harus menuliskan seluruh aturan CSS secara terpisah. Oleh karena itu, Tailwind CSS dapat dimanfaatkan dalam pembuatan landing page agar tampilan website lebih terstruktur dan dapat disesuaikan dengan kebutuhan pengembangan.

Berdasarkan uraian tersebut, praktikum ini dilakukan untuk mempelajari implementasi Next.js dan Tailwind CSS dalam pembuatan landing page website. Kegiatan dimulai dengan memeriksa instalasi Node.js, membuat aplikasi Next.js, dan menjalankan aplikasi melalui server lokal. Selanjutnya, halaman utama dimodifikasi menggunakan fungsi `Home()` dengan menerapkan Tailwind CSS untuk mengatur desain dan tata letak halaman. Melalui praktikum ini, mahasiswa diharapkan dapat memahami dasar pembuatan aplikasi web, penggunaan

komponen pada Next.js, serta penerapan Tailwind CSS dalam membangun antarmuka website sederhana.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam praktikum ini adalah sebagai berikut:

1. Bagaimana proses pembuatan aplikasi website menggunakan Next.js?
2. Bagaimana penggunaan fungsi `Home()` dalam membangun halaman utama website?
3. Bagaimana penerapan Tailwind CSS untuk memodifikasi tampilan landing page?
4. Bagaimana cara menjalankan dan menampilkan hasil landing page melalui browser menggunakan *localhost*?

1.3 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1. Memahami proses pembuatan dan menjalankan aplikasi website menggunakan Next.js.
2. Mengetahui fungsi `Home()` dalam membangun halaman utama aplikasi.
3. Mempelajari penggunaan Tailwind CSS untuk mengatur tampilan dan tata letak landing page.
4. Menghasilkan landing page sederhana yang dapat dijalankan dan ditampilkan melalui browser.

1.4 Manfaat Praktikum

Praktikum ini diharapkan dapat menambah pengetahuan dan keterampilan mahasiswa dalam pengembangan website menggunakan teknologi modern. Manfaat yang diperoleh dapat dirinci sebagai berikut:

- **Manfaat teoretis:** memperdalam pemahaman tentang konsep komponen pada React/Next.js, pendekatan *utility-first* pada Tailwind CSS, dan struktur sebuah landing page.
- **Manfaat praktis:** memberikan pengalaman langsung menulis dan memodifikasi kode, menjalankan aplikasi pada server lokal, serta memeriksa hasil pengembangan melalui browser.

- **Manfaat lanjutan:** menjadi dasar bagi mahasiswa untuk mempelajari pengembangan aplikasi web yang lebih kompleks pada praktikum selanjutnya.

1.5 Batasan Masalah

Agar pembahasan terarah, praktikum ini dibatasi pada hal-hal berikut:

- Aplikasi berupa satu halaman (*single page*) pada rute utama `/` yang ditulis pada file `app/page.tsx`.
- Konten bersifat statis; tidak menggunakan basis data, *backend*, maupun autentikasi.
- Pengembangan dan pengujian dilakukan pada sistem operasi Windows menggunakan server pengembangan lokal (`npm run dev`).
- Pengaturan tampilan sepenuhnya memakai Tailwind CSS tanpa pustaka komponen tambahan.

1.6 Sistematika Penulisan

Tabel 1.1. Sistematika Penulisan Laporan

Bab	Judul	Isi Pokok
I	Pendahuluan	Latar belakang, rumusan masalah, tujuan, manfaat, dan batasan masalah.
II	Landasan Teori dan Langkah Kerja	Teori Node.js, React, Next.js, Tailwind CSS, landing page, fungsi `Home()`, serta alat, bahan, dan langkah kerja.
III	Hasil dan Pembahasan	Proses pembuatan aplikasi, implementasi landing page, tampilan hasil, kendala, dan analisis.
IV	Penutup	Kesimpulan dan saran.

BAB II
LANDASAN TEORI DAN LANGKAH KERJA

2.1 Landasan Teori

2.1.1 Node.js dan npm

Node.js merupakan lingkungan *runtime* yang memungkinkan JavaScript dijalankan di luar browser. Node.js dibangun di atas mesin JavaScript V8 dan menggunakan model *event-driven* dengan operasi masukan/keluaran yang tidak memblokir (*non-blocking I/O*), sehingga efisien untuk menjalankan alat pengembangan dan server. Dalam pengembangan aplikasi web modern, Node.js digunakan untuk menjalankan berbagai perintah pengembangan seperti pembuatan proyek, kompilasi, dan server lokal.

Node Package Manager (npm) adalah pengelola paket bawaan Node.js. npm berfungsi mengunduh dan memasang pustaka yang dibutuhkan proyek (*dependencies*), mencatatnya pada berkas `package.json`, serta menjalankan skrip proyek seperti `npm run dev`. Perintah `npx` yang menyertai npm digunakan untuk menjalankan paket secara langsung tanpa memasangnya secara permanen, misalnya `npx create-next-app@latest`. Pada praktikum ini, versi Node.js dan npm diperiksa terlebih dahulu menggunakan perintah `node --version` dan `npm --version`.

2.1.2 JavaScript, React, dan JSX

React adalah pustaka JavaScript untuk membangun antarmuka pengguna berbasis **komponen**. Sebuah komponen adalah potongan antarmuka yang dapat digunakan ulang, ditulis sebagai fungsi JavaScript yang mengembalikan elemen tampilan. Pendekatan ini membuat halaman besar dapat dipecah menjadi bagian-bagian kecil yang lebih mudah dipelihara.

Tampilan komponen dituliskan menggunakan **JSX**, yaitu ekstensi sintaks yang menyerupai HTML tetapi ditulis di dalam JavaScript/TypeScript. Beberapa aturan penting JSX yang digunakan dalam praktikum ini adalah: (1) satu komponen harus mengembalikan satu elemen pembungkus, (2) atribut `class` ditulis `className` karena `class` adalah kata cadangan pada JavaScript, (3) ekspresi JavaScript disisipkan dengan kurung kurawal `{ }`, dan (4) setiap elemen hasil perulangan `map()` memerlukan atribut `key` yang unik. Berkas yang memuat JSX pada proyek TypeScript berekstensi `.tsx`.

2.1.3 Next.js

Next.js merupakan *framework* berbasis React yang dikembangkan oleh Vercel untuk membangun aplikasi web. Next.js menambahkan fitur siap pakai di atas React, antara lain sistem rute berbasis berkas, *rendering* di sisi server maupun statis, optimasi gambar dan huruf, serta server pengembangan dengan pembaruan cepat (*Fast Refresh*). Maharani (2025) membahas penggunaan Next.js bersama Tailwind CSS dalam pengembangan website perusahaan untuk mendukung pembuatan antarmuka yang responsif.

Next.js modern menyediakan **App Router**, yaitu sistem rute yang memanfaatkan folder `app/`. Setiap folder merepresentasikan satu segmen alamat (URL), dan berkas `page.tsx` di dalamnya menjadi halaman yang dapat diakses publik. Dengan demikian berkas `app/page.tsx` otomatis menjadi halaman utama pada alamat `/` tanpa konfigurasi rute tambahan (Next.js, 2026). Secara bawaan, komponen pada App Router adalah *Server Component*, sehingga halaman statis dapat dibangun terlebih dahulu (*prerender*) dan dikirim ke browser dengan ringan.

Tabel 2.1. File dan Folder Utama pada Proyek Next.js

File / Folder	Fungsi
`app/page.tsx`	Halaman utama (rute `/`). Berisi fungsi `Home()` yang dimodifikasi pada praktikum ini.
`app/layout.tsx`	Kerangka bersama seluruh halaman: tag `<html>`, `<body>`, dan metadata (judul, deskripsi).
`app/globals.css`	Gaya global dan impor Tailwind CSS (`@import "tailwindcss"`).
`public/`	Aset statis seperti gambar dan ikon yang dapat diakses langsung.
`package.json`	Daftar paket dan skrip proyek (`dev`, `build`, `start`).
`next.config.ts`	Konfigurasi Next.js.
`postcss.config.mjs`	Konfigurasi PostCSS yang mengaktifkan plugin Tailwind CSS.
`tsconfig.json`	Konfigurasi TypeScript.
`node_modules/`	Folder hasil instalasi seluruh paket (dihasilkan otomatis).
`.next/`	Folder keluaran *build*/server pengembangan (dihasilkan otomatis).

2.1.4 Tailwind CSS

Tailwind CSS merupakan *framework* CSS yang menggunakan pendekatan *utility-first*. Pendekatan ini memungkinkan pengembang mengatur tampilan elemen melalui kelas-kelas kecil yang masing-masing mewakili satu aturan CSS, misalnya untuk menentukan warna, ukuran teks, jarak, dan tata letak. Azhariyah dan Mukhlis (2024) membahas penerapan Tailwind CSS dalam pengembangan bagian *front-end* website dan menjelaskan pemanfaatan *utility class* untuk menyesuaikan desain serta mendukung tampilan responsif.

Pada versi terbaru, Tailwind CSS diaktifkan cukup dengan satu baris `@import "tailwindcss";` pada berkas CSS global dan plugin `@tailwindcss/postcss`. Tailwind kemudian memindai berkas kode, mengumpulkan kelas yang benar-benar dipakai, dan hanya menghasilkan CSS untuk kelas tersebut sehingga ukuran berkas gaya tetap kecil.

Tabel 2.2. Perbandingan CSS Konvensional dan Tailwind CSS

Aspek	CSS Konvensional	Tailwind CSS
Cara menulis gaya	Membuat nama *class* lalu menulis aturan di berkas CSS terpisah.	Menempelkan *utility class* langsung pada elemen.
Penamaan class	Harus menentukan nama sendiri (mis. `.tombol-utama`).	Menggunakan nama baku (mis. `rounded-lg`, `bg-blue-600`).
Berpindah berkas	Sering bolak-balik antara HTML dan CSS.	Cukup pada satu berkas komponen.
Konsistensi desain	Bergantung pada kedisiplinan penulis.	Terjaga oleh skala warna, ukuran, dan jarak bawaan.
Responsif	Menulis `@media` secara manual.	Cukup menambah awalan seperti `md:`.

Beberapa kelompok *utility class* yang paling sering digunakan dirangkum pada Tabel 2.3 berikut. Satu satuan skala jarak Tailwind setara 0,25 rem (sekitar 4 piksel), sehingga `px-6` berarti *padding* horizontal 1,5 rem atau sekitar 24 piksel.

Tabel 2.3. Kelompok Utility Class Tailwind CSS

Kelompok	Contoh Class	Fungsi
Warna latar	`bg-white`, `bg-blue-600`, `bg-slate-950`	Mengatur warna latar belakang.
Warna teks	`text-white`, `text-slate-700`	Mengatur warna teks.
Tipografi	`text-4xl`, `font-bold`, `leading-7`, `uppercase`	Ukuran, ketebalan, tinggi baris, dan gaya huruf.

Kelompok	Contoh Class	Fungsi
Tata letak	`flex`, `grid`, `grid-cols-3`, `items-center`	Mengatur susunan elemen dengan Flexbox atau Grid.
Jarak	`p-6`, `px-6`, `py-3`, `mt-4`, `gap-6`	Padding, margin, dan celah antarelemen.
Ukuran	`w-full`, `max-w-6xl`, `h-10`, `min-h-screen`	Lebar dan tinggi elemen.
Bentuk dan efek	`rounded-2xl`, `shadow-lg`, `border`, `backdrop-blur`	Sudut, bayangan, garis tepi, dan efek visual.
Interaksi	`hover:bg-blue-500`, `transition`	Perubahan tampilan saat kursor diarahkan dan animasi halus.

Untuk mendukung tampilan responsif, Tailwind menyediakan awalan *breakpoint* dengan prinsip *mobile-first*: kelas tanpa awalan berlaku pada semua ukuran layar, sedangkan kelas berawalan (misalnya `md:`) berlaku mulai ukuran layar tertentu ke atas. Contohnya, `hidden md:flex` berarti elemen disembunyikan pada layar kecil dan ditampilkan sebagai Flexbox mulai layar menengah. Daftar *breakpoint* disajikan pada Tabel 2.4.

Tabel 2.4. Breakpoint Responsif Tailwind CSS

Awalan	Lebar Minimum	Perangkat Umum
(tanpa awalan)	0 px	Semua ukuran, termasuk ponsel
`sm:`	640 px (40 rem)	Ponsel besar / layar kecil
`md:`	768 px (48 rem)	Tablet
`lg:`	1024 px (64 rem)	Laptop
`xl:`	1280 px (80 rem)	Desktop
`2xl:`	1536 px (96 rem)	Layar sangat lebar

2.1.5 Landing Page

Landing page merupakan halaman website yang dirancang untuk menyampaikan informasi utama kepada pengunjung dan mengarahkan mereka pada tindakan tertentu. Penyusunan bagian-bagian halaman perlu memperhatikan keterbacaan dan kemudahan navigasi agar informasi dapat dipahami dengan baik. Bagian-bagian yang lazim terdapat pada landing page ditunjukkan pada Tabel 2.5.

Tabel 2.5. Bagian-Bagian Umum Landing Page

Bagian	Fungsi
Navigasi (*navbar*)	Menyediakan tautan menuju bagian-bagian halaman dan identitas (logo).
Hero section	Bagian pembuka berisi judul utama, deskripsi singkat, dan tombol ajakan bertindak (*call to action*).
Bagian konten / fitur	Menjelaskan informasi pokok yang ingin disampaikan, biasanya dalam bentuk kartu.
Testimoni / kutipan	Memperkuat pesan atau memberi sentuhan personal.
Footer	Penutup halaman berisi hak cipta dan informasi tambahan.

Prinsip desain yang diperhatikan meliputi hierarki visual (judul lebih menonjol daripada isi), kontras warna yang cukup agar teks terbaca, penggunaan ruang kosong (*whitespace*) yang memadai, konsistensi warna dan bentuk, serta tampilan responsif agar nyaman dilihat pada berbagai perangkat.

2.1.6 Fungsi Home()

Dalam Next.js, fungsi `Home()` digunakan sebagai komponen yang menghasilkan tampilan halaman utama. Fungsi tersebut harus diekspor secara bawaan (`export default`) agar dikenali sebagai halaman, dan mengembalikan elemen antarmuka melalui pernyataan `return` yang kemudian ditampilkan oleh aplikasi. Pada struktur proyek yang menggunakan App Router, halaman utama ditulis pada berkas `app/page.tsx`. Melalui berkas tersebut, pengembang dapat menambahkan elemen seperti judul, paragraf, tombol, dan bagian informasi dengan menggunakan JSX serta *class* Tailwind CSS. Nama fungsi `Home` hanyalah konvensi; yang menentukan sebuah halaman adalah nama berkas `page.tsx` beserta ekspor bawaannya.

2.1.7 Server Pengembangan Lokal

Perintah `npm run dev` menjalankan server pengembangan Next.js pada komputer sendiri. Server ini diakses melalui alamat *localhost* (`127.0.0.1`), yaitu alamat *loopback* yang merujuk pada komputer itu sendiri, diikuti nomor *port*. Port bawaan Next.js adalah 3000, sehingga alamat standar aplikasi adalah `http://localhost:3000`. Apabila port tersebut sedang digunakan proses lain, Next.js otomatis memilih port kosong berikutnya dan menampilkannya pada terminal. Server pengembangan juga melakukan *Fast Refresh*: setiap perubahan yang disimpan langsung tercermin di browser tanpa memuat ulang halaman secara manual.

2.1.8 Visual Studio Code

Visual Studio Code (VS Code) adalah editor kode sumber yang digunakan untuk menulis dan mengelola berkas proyek. VS Code menyediakan panel *Explorer* untuk melihat struktur folder, terminal terintegrasi untuk menjalankan perintah, serta penyorotan sintaks untuk TypeScript dan JSX sehingga memudahkan penulisan kode.

2.1.9 Penelitian Terkait

Tabel 2.6. Rujukan Penelitian yang Mendasari Praktikum

Peneliti	Fokus Pembahasan	Relevansi dengan Praktikum
Maharani (2025)	Pengembangan website PT. Rantangin Digital Indonesia dengan Next.js dan Tailwind CSS.	Menunjukkan kombinasi Next.js dan Tailwind CSS untuk membangun antarmuka website yang responsif.
Azhariyah dan Mukhlis (2024)	Penggunaan Tailwind CSS pada pengembangan bagian *front-end* website.	Menjelaskan pemanfaatan *utility class* untuk desain dan tampilan responsif.
Next.js (2026)	Dokumentasi resmi Next.js.	Rujukan fungsi `app/page.tsx` pada App Router dan cara menjalankan server pengembangan.

2.2 Alat dan Bahan

Alat dan bahan yang digunakan dalam praktikum ini dirangkum pada Tabel 2.7.

Tabel 2.7. Alat dan Bahan Praktikum

No	Alat / Bahan	Keterangan	Fungsi
1	Laptop / komputer	Sistem operasi Windows	Perangkat untuk menjalankan seluruh praktikum.
2	Node.js	Versi LTS terbaru	Lingkungan *runtime* JavaScript.
3	npm	Bawaan Node.js	Pengelola paket dan penjalan skrip proyek.
4	Visual Studio Code	Editor kode	Menulis dan mengedit kode program.
5	Next.js	Versi terbaru (create-next-app@latest)	*Framework* pengembangan website.

No	Alat / Bahan	Keterangan	Fungsi
6	Tailwind CSS	Dipilih saat pembuatan proyek	Mengatur desain antarmuka.
7	Browser	Google Chrome	Melihat dan menguji tampilan website.

2.3 Langkah Kerja

2.3.1 Pemeriksaan Node.js

Langkah pertama adalah memastikan Node.js telah terinstal pada komputer. Pemeriksaan dilakukan dengan membuka Command Prompt atau terminal, kemudian menjalankan perintah berikut. Jika nomor versi muncul, berarti Node.js dan npm siap digunakan.

Terminal
```
node --version
npm --version
```

2.3.2 Pembuatan Project Next.js

Setelah Node.js tersedia, terminal diarahkan ke folder praktikum `pertemuan-6`. Selanjutnya, aplikasi dibuat menggunakan perintah berikut:

Terminal
```
npx create-next-app@latest landing-page
```

Perintah tersebut membuat proyek Next.js baru dengan nama `landing-page`. Pada proses konfigurasi, Tailwind CSS dan App Router dipilih agar sesuai dengan kebutuhan praktikum. Pertanyaan konfigurasi dapat sedikit berbeda antarversi; jawaban lain dapat mengikuti pilihan bawaan.

2.3.3 Membuka Project di Visual Studio Code

Setelah proses instalasi selesai, terminal diarahkan ke folder aplikasi, lalu proyek dibuka menggunakan Visual Studio Code. Pada tahap ini, berkas dan struktur folder proyek diperiksa untuk menemukan berkas halaman utama yang akan dimodifikasi.

Terminal
```
cd landing-page
code .
```

2.3.4 Menjalankan Aplikasi

Aplikasi dijalankan melalui terminal Visual Studio Code. Setelah server pengembangan aktif, browser dibuka dan diarahkan ke alamat `http://localhost:3000`. Tahap ini bertujuan memastikan bahwa aplikasi dapat berjalan dan halaman awal dapat ditampilkan dengan baik.

```
Terminal
npm run dev
```

2.3.5 Modifikasi Halaman Landing Page

Tahap berikutnya adalah membuka berkas `app/page.tsx` dan memodifikasi fungsi `Home()` menjadi landing page. Modifikasi dilakukan dengan menambahkan bagian navigasi, judul utama, deskripsi, tombol, informasi konten, dan *footer*. Tailwind CSS digunakan untuk mengatur warna, ukuran teks, jarak, tata letak, serta efek tampilan pada elemen halaman.

2.3.6 Pemeriksaan Hasil

Setelah kode selesai dimodifikasi, berkas disimpan dan hasilnya diperiksa melalui browser. Pemeriksaan dilakukan untuk memastikan seluruh bagian landing page tampil sesuai dengan kode yang dibuat. Apabila terdapat kesalahan atau tampilan yang belum sesuai, kode diperbaiki dan browser diperiksa kembali hingga halaman dapat ditampilkan dengan baik.

Tabel 2.8. Ringkasan Perintah yang Digunakan

No	Perintah	Kegunaan
1	`node --version`	Memeriksa versi Node.js.
2	`npm --version`	Memeriksa versi npm.
3	`npx create-next-app@latest landing-page`	Membuat proyek Next.js bernama *landing-page*.
4	`cd landing-page`	Masuk ke folder proyek.
5	`code .`	Membuka folder proyek di Visual Studio Code.
6	`npm run dev`	Menjalankan server pengembangan.
7	`npm run build`	Membuat *build* produksi (opsional, untuk pengecekan).

BAB III
HASIL DAN PEMBAHASAN

3.1 Hasil Praktikum

Praktikum pembuatan landing page menggunakan Next.js dan Tailwind CSS bertujuan menerapkan materi yang telah dipelajari dalam pengembangan website. Tahapan praktikum meliputi pemeriksaan Node.js, pembuatan proyek Next.js, pengeditan halaman utama, serta pengujian aplikasi melalui browser. Next.js menggunakan sistem berbasis berkas untuk menentukan halaman aplikasi, sehingga berkas `app/page.tsx` digunakan untuk membangun halaman utama pada proyek yang menggunakan App Router (Next.js, 2026).

Hasil akhir praktikum adalah sebuah landing page perkenalan diri bernama **Fadhlun** dengan tema gelap bernuansa biru dan *slate* yang bersih serta profesional. Halaman terdiri atas bagian navigasi, *hero section* yang memuat judul dan deskripsi, bagian *Tentang Saya*, bagian *Hobi*, kutipan motivasi, serta *footer*. Pengaturan tampilan dilakukan menggunakan Tailwind CSS melalui *utility class* yang diterapkan pada elemen JSX, sehingga desain halaman dapat dimodifikasi langsung melalui kode program tanpa menuliskan seluruh aturan CSS secara terpisah.

3.2 Proses Pembuatan Aplikasi

3.2.1 Pemeriksaan Node.js

Sebelum membuat aplikasi, dilakukan pemeriksaan untuk memastikan Node.js dan npm tersedia pada komputer. Pemeriksaan dilakukan melalui terminal dengan perintah `node --version` dan `npm --version`. Hasilnya ditampilkan pada Gambar 3.1 dan dirangkum pada Tabel 3.1 (isi versi sesuai hasil pada komputer masing-masing).

Tabel 3.1. Hasil Pemeriksaan Lingkungan Pengembangan

Perangkat Lunak	Perintah	Hasil
Node.js	`node --version`	hasil
npm	`npm --version`	hasil
Next.js	Terlihat saat `npm run dev`	hasil

Node.js digunakan untuk menjalankan lingkungan pengembangan JavaScript, sedangkan npm digunakan untuk mengelola paket serta menjalankan perintah proyek. Karena kedua versi muncul, lingkungan dinyatakan siap dan proses dapat dilanjutkan.

3.2.2 Pembuatan Project Next.js

Setelah pemeriksaan selesai, terminal diarahkan ke folder praktikum `pertemuan-6`. Aplikasi kemudian dibuat menggunakan perintah `npx create-next-app@latest landing-page`. Pada proses konfigurasi, Tailwind CSS dan App Router dipilih agar proyek sesuai dengan kebutuhan praktikum.

Setelah proses pembuatan proyek selesai, folder aplikasi dibuka menggunakan Visual Studio Code. Struktur folder diperiksa untuk menemukan berkas `app/page.tsx`, yang menjadi tempat penulisan kode halaman utama (Gambar 3.2). Tahap ini memberikan pemahaman mengenai struktur dasar proyek Next.js dan lokasi berkas yang perlu dimodifikasi untuk mengubah tampilan halaman.

Pada panel *Explorer* terlihat folder `app` berisi `favicon.ico`, `globals.css`, `layout.tsx`, dan `page.tsx`, ditambah folder `public`, `node_modules`, serta berkas konfigurasi seperti `next.config.ts`, `package.json`, dan `tsconfig.json` sebagaimana dijelaskan pada Tabel 2.1.

3.2.3 Menjalankan Aplikasi

Aplikasi dijalankan melalui terminal Visual Studio Code menggunakan perintah `npm run dev`. Setelah server pengembangan aktif, browser dibuka melalui alamat `http://localhost:3000`. Alamat tersebut digunakan untuk menampilkan halaman aplikasi yang berjalan pada komputer lokal. Dokumentasi resmi Next.js juga menjelaskan bahwa aplikasi dapat dijalankan menggunakan perintah pengembangan dan diperiksa melalui browser pada alamat lokal tersebut (Next.js, 2026).

Keluaran terminal pada Gambar 3.3 menampilkan versi Next.js, alamat *Local* dan *Network* tempat aplikasi berjalan, serta status *Ready*. Alamat pada baris *Local* itulah yang dibuka di browser. Apabila port 3000 sedang dipakai proses lain, Next.js otomatis memakai port berikutnya, dan pesan peringatan tertentu dapat muncul; hal tersebut dibahas pada subbab 3.4.2.

3.3 Implementasi Landing Page

3.3.1 Pembuatan Fungsi Home()

Modifikasi halaman utama dilakukan pada berkas `app/page.tsx` dengan menggunakan fungsi `Home()`. Fungsi ini merupakan komponen yang menghasilkan elemen antarmuka untuk ditampilkan pada halaman utama website. Bagian `return` digunakan untuk mengembalikan struktur JSX yang berisi elemen-elemen halaman, seperti judul, paragraf, tombol, dan bagian informasi lainnya.

Agar kode rapi, data yang berulang (tautan navigasi, kartu *Tentang Saya*, dan kartu hobi) disimpan dalam larik (*array*) di luar fungsi, kemudian ditampilkan dengan `map()`. Dengan cara ini, menambah satu kartu cukup dengan menambah satu objek pada larik.

app/page.tsx (cuplikan data)

```
const navLinks = [
  { label: "Beranda", href: "#beranda" },
  { label: "Tentang Saya", href: "#tentang" },
  { label: "Hobi", href: "#hobi" },
  { label: "Motivasi", href: "#motivasi" },
];
```

app/page.tsx (kerangka fungsi Home)

```
export default function Home() {
  return (
    <main className="min-h-screen bg-[#0a0f1a]
                     text-slate-200">
      {/* Navigasi */}
      {/* Hero Section */}
      {/* Tentang Saya */}
      {/* Hobi */}
      {/* Motivasi */}
      {/* Footer */}
    </main>
  );
}
```

Penggunaan fungsi `Home()` membantu menyusun halaman utama secara terstruktur. Kode di dalam fungsi tersebut dapat dimodifikasi untuk menambahkan atau mengubah elemen sesuai kebutuhan desain. Dengan demikian, mahasiswa dapat memahami hubungan antara kode program yang ditulis dengan tampilan website yang dihasilkan pada browser.

3.3.2 Penerapan Tailwind CSS

Tailwind CSS diterapkan dengan menambahkan *utility class* pada atribut `className` elemen JSX. Class seperti `bg-slate-950` digunakan untuk mengatur warna latar, `text-white` untuk warna teks, `text-4xl` untuk ukuran teks, dan `font-bold` untuk membuat teks tebal. Selain itu, class seperti `flex`, `grid`, `gap-6`, dan `px-6` digunakan untuk mengatur tata letak serta jarak antarelemen. Penerapan pada tombol dan navigasi ditunjukkan pada cuplikan berikut.

app/page.tsx (cuplikan navigasi)

```
<header className="sticky top-0 z-50 border-b border-slate-800/80
                   bg-[#0a0f1a]/85 backdrop-blur">
  <nav className="mx-auto flex max-w-6xl items-center
                  justify-between px-6 py-4">
    <a href="#beranda" className="text-lg font-bold
                                  text-white">
      fadhlun<span className="text-blue-500">.dev</span>
    </a>
    <ul className="hidden items-center gap-9 md:flex">
      ...
    </ul>
  </nav>
</header>
```

app/page.tsx (cuplikan tombol pada hero)

```
<a href="#tentang"
   className="rounded-md bg-blue-600 px-6 py-3 text-sm
              font-semibold text-white shadow-lg
              shadow-blue-950/50 transition
              hover:bg-blue-500">
  Tentang Saya
</a>
```

Contoh di atas memperlihatkan keunggulan pendekatan *utility-first*: satu baris `className` sudah menentukan warna, bentuk membulat, jarak dalam, ketebalan huruf, bayangan, dan efek *hover*. Penerapan *utility class* tersebut memudahkan pengaturan desain karena perubahan tampilan dapat dilakukan langsung pada elemen yang bersangkutan.

Tailwind CSS juga menyediakan variasi class responsif. Pada kode navigasi, `hidden md:flex` menyembunyikan menu pada layar kecil dan menampilkannya pada layar menengah ke atas, sedangkan `grid gap-5 md:grid-cols-2` membuat kartu tersusun satu kolom pada layar kecil dan dua kolom pada layar lebih lebar. Ringkasan class yang dipakai pada setiap bagian halaman terdapat pada Tabel 3.2.

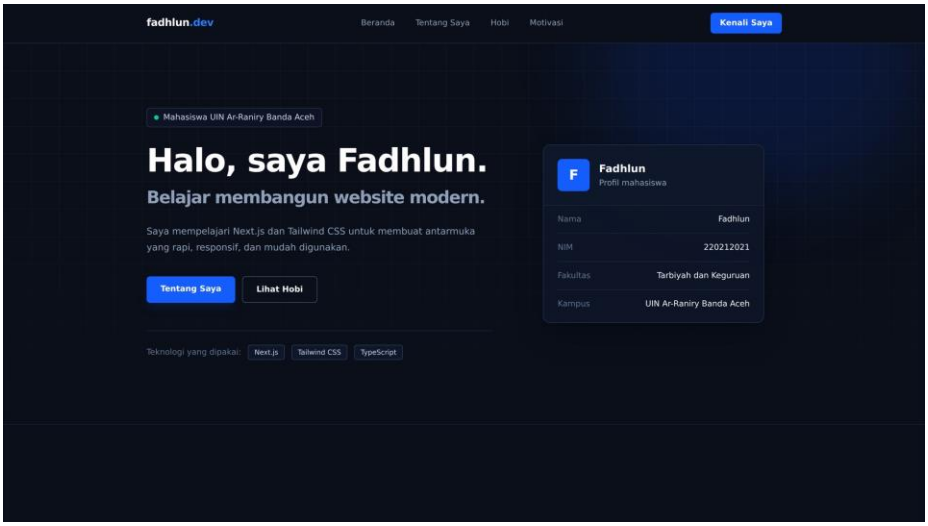
Tabel 3.2. Class Tailwind CSS pada Tiap Bagian Landing Page

Bagian	Class Utama	Efek yang Dihasilkan
Navigasi	`sticky top-0`, `backdrop-blur`, `flex`, `justify-between`	Menu menempel di atas halaman dengan latar transparan buram; logo di kiri, tombol di kanan.
Hero	`md:grid-cols-[1.15fr_ 0.85fr]`, `md:text-[3.5rem]`, `bg-grid`	Dua kolom (teks dan kartu) pada layar lebar; judul membesar; latar berpola garis tipis.
Kartu profil	`rounded-xl`, `border border-slate-700/70`, `divide-y`, `shadow-2xl`	Kartu berbingkai tipis berisi data diri (nama, NIM, fakultas, kampus) dengan pemisah antarbaris.
Tentang Saya	`grid gap-5 md:grid-cols-2`, `rounded-lg`, `border-l-2 border-l-blue-500`	Empat kartu dua kolom dengan garis aksen biru di sisi kiri.
Hobi	`grid md:grid-cols-3`, `text-blue-400`	Tiga kartu hobi sejajar dengan ikon biru.
Motivasi	`max-w-4xl`, `border-l-4`, `border-blue-500`, `pl-8`	Kutipan besar dengan garis biru di sisi kiri.
Footer	`border-t`, `py-8`, `flex justify-between`, `text-sm`	Penutup; hak cipta di kiri dan kampus di kanan.

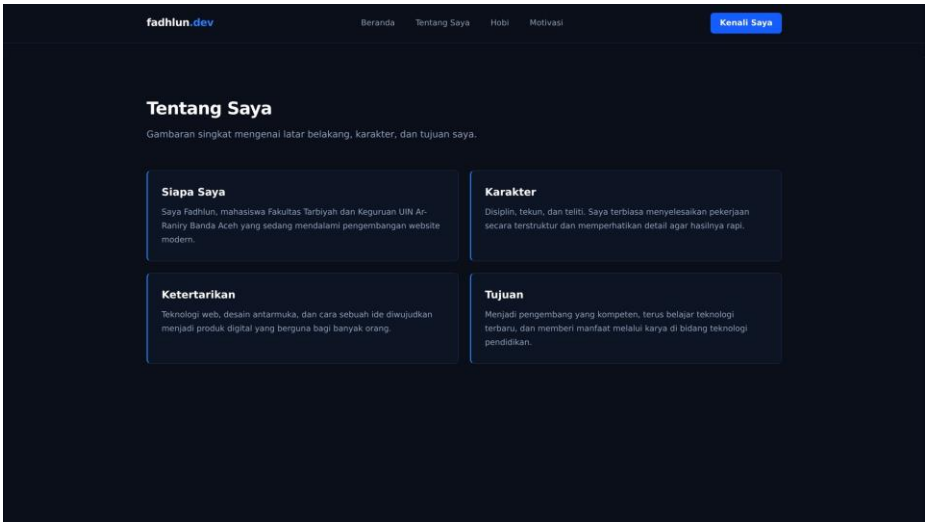
3.3.3 Struktur Halaman Landing Page

Landing page yang dibuat terdiri atas beberapa bagian utama. Bagian navigasi membantu pengunjung berpindah ke bagian tertentu pada halaman melalui tautan *anchor* (`#beranda`, `#tentang`, `#hobi`, `#motivasi`). Bagian utama atau *hero section* menampilkan judul, deskripsi, dan tombol yang memperkenalkan tujuan website. Selanjutnya, bagian *Tentang Saya* dan *Hobi* menjelaskan informasi utama yang ingin disampaikan kepada pengunjung. Bagian penutup atau *footer* ditempatkan di akhir halaman untuk memberikan informasi tambahan.

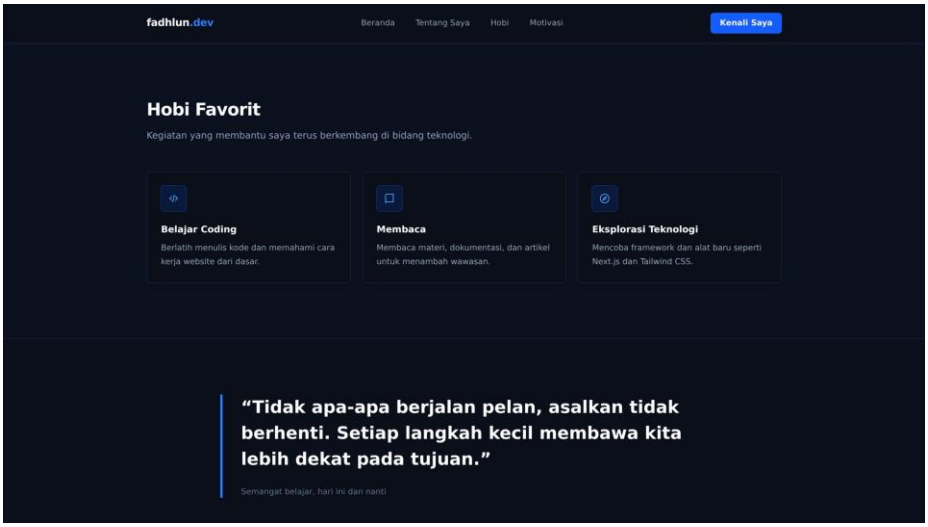
Susunan tersebut membentuk halaman dengan alur informasi yang jelas. Penggunaan warna, ukuran teks, jarak, dan tata letak melalui Tailwind CSS digunakan untuk membedakan setiap bagian. Dengan demikian, landing page tidak hanya berfungsi sebagai halaman pembuka, tetapi juga sebagai sarana latihan menerapkan prinsip dasar penyusunan antarmuka website. Hasil tampilan pada browser ditunjukkan pada Gambar 3.4 sampai Gambar 3.6.



Gambar 3.4. Tampilan navigasi dan hero section pada browser



Gambar 3.5. Tampilan bagian Tentang Saya pada browser



Gambar 3.6. Tampilan bagian Hobi dan kutipan motivasi pada browser

3.4 Pembahasan

3.4.1 Analisis Hasil

Berdasarkan tahapan praktikum, pembuatan landing page menggunakan Next.js dan Tailwind CSS memberikan pengalaman dalam memahami proses dasar pengembangan website. Next.js digunakan untuk membangun halaman melalui komponen, sedangkan Tailwind CSS digunakan untuk mengatur tampilan visual halaman. Penggunaan kedua teknologi tersebut memungkinkan struktur halaman dan desain antarmuka dikembangkan dalam satu proyek secara terorganisasi.

Fungsi `Home()` menjadi bagian penting dalam praktikum karena digunakan untuk menghasilkan halaman utama. Melalui fungsi tersebut, elemen JSX dapat disusun sesuai kebutuhan dan diberi *utility class* Tailwind CSS untuk menentukan

tampilannya. Proses menjalankan aplikasi melalui `npm run dev` juga memberikan pemahaman mengenai penggunaan server pengembangan lokal untuk memeriksa perubahan kode secara langsung melalui browser.

Kesesuaian hasil dengan rancangan diperiksa berdasarkan tampilan pada browser dan dirangkum pada Tabel 3.3.

Tabel 3.3. Kesesuaian Hasil dengan Rancangan Landing Page

Bagian yang Diperiksa	Rancangan	Hasil pada Browser
Aplikasi berjalan	Dapat diakses pada `localhost`	Halaman tampil pada `localhost:3000` (Gambar 3.4)
Navigasi	Logo, empat tautan menu, dan tombol *Kenali Saya*	Tampil sesuai (Gambar 3.4)
Hero section	Judul, deskripsi, dua tombol, daftar teknologi, dan kartu profil	Tampil sesuai (Gambar 3.4)
Tentang Saya	Empat kartu beraksen biru pada latar gelap	Tampil dua kolom (Gambar 3.5)
Hobi	Tiga kartu hobi berikon	Tampil tiga kolom (Gambar 3.6)
Motivasi	Kutipan besar bergaris biru	Tampil sesuai (Gambar 3.6)

3.4.2 Kendala dan Penyelesaian

Dalam praktikum ini terdapat beberapa pesan pada terminal yang mungkin muncul dan perlu dipahami. Kendala beserta penyelesaiannya dirangkum pada Tabel 3.4.

Tabel 3.4. Kendala yang Ditemui dan Penyelesaiannya

Kendala	Penyebab	Penyelesaian
Port 3000 sedang digunakan; aplikasi pindah ke port lain	Sebuah proses lain (mis. server `next dev` sebelumnya) masih memakai port 3000.	Gunakan alamat pada baris *Local:* di terminal, atau hentikan proses lama lalu jalankan ulang `npm run dev`.
Pesan *Another next dev server is already running*	Server pengembangan untuk proyek yang sama sudah aktif pada port 3001.	Akses server yang ada, atau hentikan dengan `taskkill /PID <nomor> /F` lalu mulai ulang.
Peringatan *inferred your workspace root*	Terdapat lebih dari satu `package-lock.json` pada folder induk (proyek bersarang).	Hapus lockfile yang tidak diperlukan atau atur `turbopack.root` pada `next.config.ts`; peringatan tidak menghentikan aplikasi.

Kendala	Penyebab	Penyelesaian
Perubahan kode belum tampil	Berkas belum disimpan atau server berhenti.	Simpan berkas (Ctrl+S) dan pastikan `npm run dev` masih berjalan.

3.4.3 Kelebihan dan Keterbatasan

- **Kelebihan:** struktur berbasis komponen membuat kode rapi; Tailwind CSS mempercepat penyesuaian desain karena gaya ditulis di elemen yang sama; server pengembangan menampilkan perubahan secara langsung; dan class responsif memudahkan adaptasi ke berbagai ukuran layar.
- **Keterbatasan:** `className` dapat menjadi panjang bila elemen memiliki banyak gaya; pemula perlu menghafal nama class; dan halaman masih statis tanpa data dinamis maupun *backend*.

3.4.4 Jawaban atas Rumusan Masalah

Tabel 3.5. Jawaban atas Rumusan Masalah

No	Rumusan Masalah	Jawaban Ringkas
1	Proses pembuatan aplikasi dengan Next.js	Memeriksa Node.js dan npm, lalu menjalankan `npx create-next-app@latest landing-page` dengan memilih Tailwind CSS dan App Router.
2	Penggunaan fungsi `Home()`	Komponen pada `app/page.tsx` yang diekspor secara bawaan dan mengembalikan JSX sebagai tampilan halaman utama.
3	Penerapan Tailwind CSS	Menambahkan *utility class* pada `className` untuk warna, tipografi, tata letak, jarak, efek, dan tampilan responsif (`md:`).
4	Menjalankan dan menampilkan hasil	Menjalankan `npm run dev`, lalu membuka `http://localhost:3000` di browser.

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Next.js merupakan *framework* berbasis React yang dapat digunakan untuk membangun aplikasi website. Dalam praktikum ini, Node.js digunakan sebagai lingkungan untuk menjalankan perintah pengembangan, sedangkan Next.js digunakan untuk membuat proyek website. Tailwind CSS dimanfaatkan untuk mengatur tampilan halaman melalui penggunaan class pada elemen JSX.

Melalui praktikum ini, mahasiswa dapat memahami tahapan dasar pembuatan website menggunakan Next.js, mulai dari memeriksa instalasi Node.js, membuat proyek, membuka proyek melalui Visual Studio Code, menjalankan server pengembangan, hingga memodifikasi halaman utama melalui berkas `app/page.tsx` dengan fungsi `Home()`. Hasil praktikum berupa landing page perkenalan diri berhasil ditampilkan melalui browser pada alamat lokal `http://localhost:3000` dan terdiri atas navigasi, *hero section*, bagian *Tentang Saya*, bagian *Hobi*, kutipan motivasi, serta *footer*.

4.2 Saran

Dalam pelaksanaan praktikum selanjutnya, disarankan hal-hal berikut:

1. Mahasiswa lebih memahami struktur folder dan fungsi setiap berkas dalam proyek Next.js (Tabel 2.1).
2. Mahasiswa berlatih menggunakan Tailwind CSS agar dapat membuat tampilan website yang lebih menarik, rapi, dan responsif, misalnya dengan menguji tampilan pada berbagai ukuran layar.
3. Pemeriksaan hasil melalui browser dilakukan setiap kali terdapat perubahan kode agar kesalahan dapat diketahui dan diperbaiki lebih awal.
4. Hentikan server pengembangan yang tidak terpakai sebelum menjalankan `npm run dev` kembali agar tidak terjadi bentrok port.
5. Pengembangan berikutnya dapat memecah halaman menjadi beberapa komponen terpisah dan menambahkan fitur dinamis, seperti formulir kontak atau data dari *API*.

DAFTAR PUSTAKA

Azhariyah, & Mukhlis. (2024). *[lengkapi judul artikel, nama jurnal/prosiding, volume, halaman, dan DOI sesuai sumber asli]*

Maharani. (2025). *[lengkapi judul penelitian mengenai pengembangan website PT. Rantangin Digital Indonesia, penerbit/institusi, dan tautan sumber]*

Next.js. (2026). *Next.js Documentation: App Router*. Vercel. https://nextjs.org/docs

Tailwind Labs. (2026). *Tailwind CSS Documentation*. https://tailwindcss.com/docs

Meta Platforms. (2026). *React Documentation*. https://react.dev

OpenJS Foundation. (2026). *Node.js Documentation*. https://nodejs.org/docs

Microsoft. (2026). *Visual Studio Code Documentation*. https://code.visualstudio.com/docs